PAYMENTS & OPERATIONS PLAN

Paying operators for 2027 bookings

A plain-language plan for the new Operator Payments module — how NAT passes on the money it holds, accurately, every time, in a way anyone on the team can follow.

Money NAT holds → operator's bank

Every number explainable

Preview before real money

Prepared by
Devat Karetha
Solutions Lead

Date
October 6, 2026 · updated

Status
All decisions made · ready to build

01 — THE SHORT VERSION

What this plan is about

From 2027, when a guest pays by card, the whole amount stays in NAT's Stripe balance. The tour operator is not paid automatically any more. **Today, nothing sends operators that money yet.** This plan builds the part that does.

THE PROBLEM

Auto Payout only knows how to pay out money that already sits in the operator's own Stripe account. 2027 money never gets there, so Auto Payout deliberately skips it. Without a new module, 2027 operators would never be paid.

THE ANSWER

A new module, **Operator Payments**. It finds every 2027 payment that is ready, works out exactly what each operator is owed, lets an admin approve it, moves the money to the operator's Stripe account, then sends it to their bank.

What makes it trustworthy

- **Each payment is counted once.** The system itself refuses to pay the same payment twice.
- **One calculation, same as today.** It gives the same answer as Auto Payout for the same bookings — we test this on purpose.
- **Refunds and disputes are handled automatically** — before or after the operator is paid.
- **Preview mode first.** We run it for several cycles without moving any real money and compare the results.
- **NAT keeps only its own fee.** All ticket cost goes to the operator — no safety deposit and no minimum balance is held back.
- **Built for non-technical staff:** plain words, a "Why this amount?" explanation on every line, and clear next steps.

ALREADY DECIDED

Two bank deposits in version 1 · pay by tour date *or* purchase date, chosen per operator · NAT keeps only its fee — the operator is owed *payment − NAT fee*, for every booking including partner bookings · refunds after payment are deducted from the operator's next payment · disputes handled only when lost, only for operators with the Dispute Reversal setting · one Stripe account per operator, with the code ready for package-level accounts later · **Stripe test (sandbox) mode only until you say go-live.**

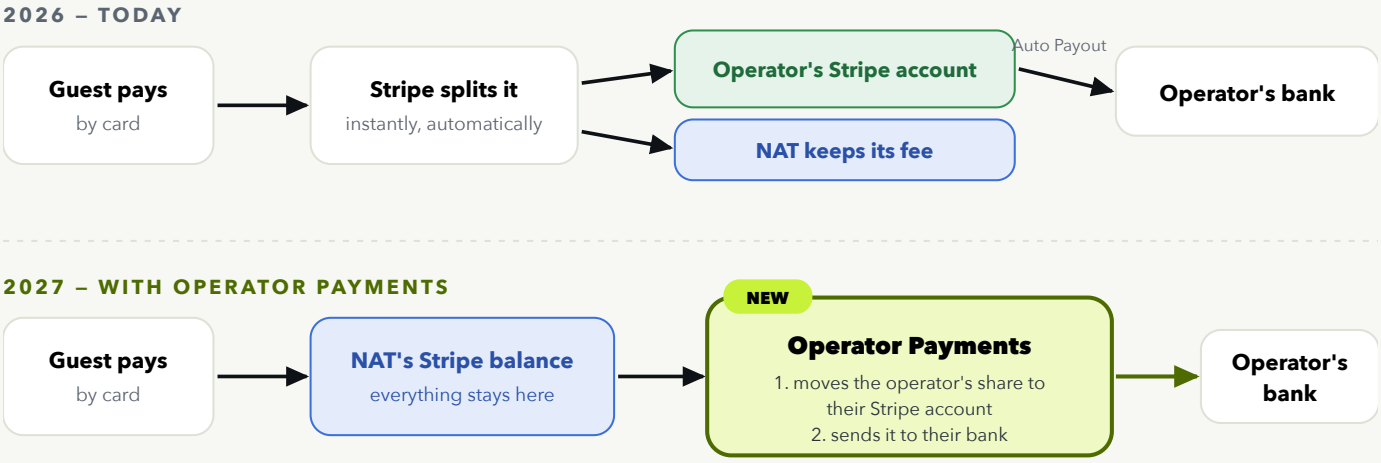
NOTHING LEFT TO DECIDE

The safety hold and safety balance are dropped, and the operator's own Stripe account is the payee for every tenant. The next step is your go-ahead for Phase 1.

02 — WHAT CHANGES IN 2027

Where the money goes

Until the end of 2026 Stripe splits every card payment the moment it happens. In 2027 it doesn't — so something has to carry the money onward.



NAT's own fee is simply left in NAT's balance — it is never sent to the operator.
Auto Payout keeps working exactly as it does today for 2026 bookings. It is not changed to carry 2027 money.

Fig. 1 — The 2027 flow adds one new step between NAT's balance and the operator's bank.

Why not just extend Auto Payout?

Auto Payout pays from the operator's *own* Stripe balance, and it tracks "paid / not paid" for a whole booking. 2027 money needs an extra step first (moving it into that balance), and it is safer to keep the tried-and-tested 2026 payout untouched. So the new module sits beside it, **reuses its rules** (schedule, waiting days, minimum and maximum amounts, notification emails, approval) and **follows its calculation** — but keeps its own records.

> **IN PLAIN WORDS**
> Operators will see two deposits at first: the usual Auto Payout, and a separate one labelled for 2027 bookings. They can be merged into one later, once everything has proven itself.

03 — HOW A PAYMENT RUN WORKS

From "ready" to "in the bank"

Each time the schedule comes round, the module prepares a **payment run** for the operator. Nothing real happens until an admin approves it.

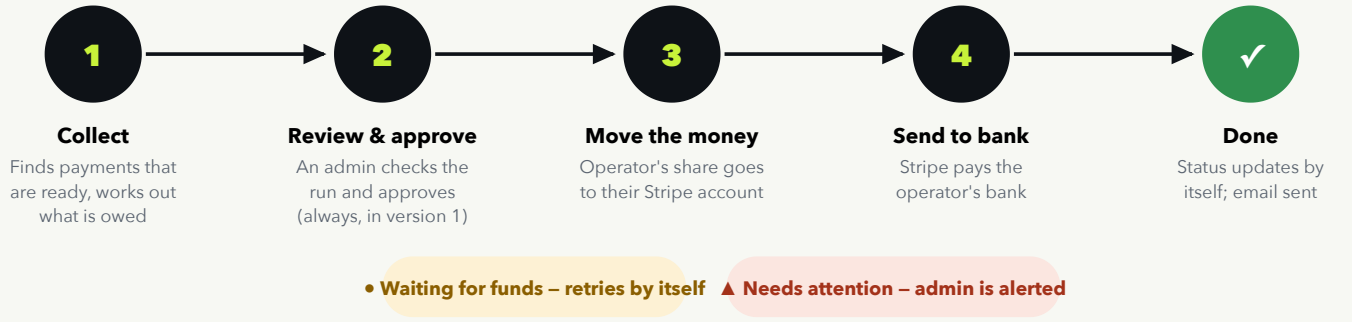


Fig. 2 – A payment run in five steps, plus the two "pause" states. The words in the figure are the words staff will see on screen.

What the screen will call each stage

STATUS	WHAT IT MEANS	WHAT THE ADMIN DOES
● Waiting for approval	The run is prepared and checked. No money has moved.	Review, then approve or reject.
▶ Paying	Approved. Money is being moved and sent to the bank.	Nothing – it runs by itself.
● Waiting for funds	The money isn't available in NAT's Stripe balance yet (recent card payments take a few days to clear).	Nothing. It retries automatically.
▲ Needs attention	Something couldn't be completed, e.g. the operator has no bank account on file.	Follow the plain-language hint shown.
■ Paid	Stripe confirmed the bank payment.	Nothing.

If something goes wrong halfway

Moving the money and sending it to the bank are **two separate steps, each remembered separately**. If the first works and the second fails, the money simply waits safely in the operator's Stripe account and only the bank step is retried. **The money is never moved twice.**

> **TO BE PROVEN, NOT ASSUMED**
> How quickly moved money becomes available for the bank payout depends on Stripe. We will measure it in Stripe's test mode before any real operator is paid; if it isn't instant, the run simply waits and retries.

04 — WHEN DOES AN OPERATOR GET PAID?

Two ways to decide the date

Each operator gets one setting: pay by **tour date** or by **purchase date**, plus a number of days to wait.

TOUR DATE + DAYS

A payment becomes ready a few days *after the tour happens*. This is how Auto Payout works today. Safest for refunds, slower for the operator.

PURCHASE DATE + DAYS

A payment becomes ready a few days *after the guest paid*, even if the tour is months away. Faster for the operator. If the guest is refunded later, it is **deducted from the operator's next payment** (agreed).

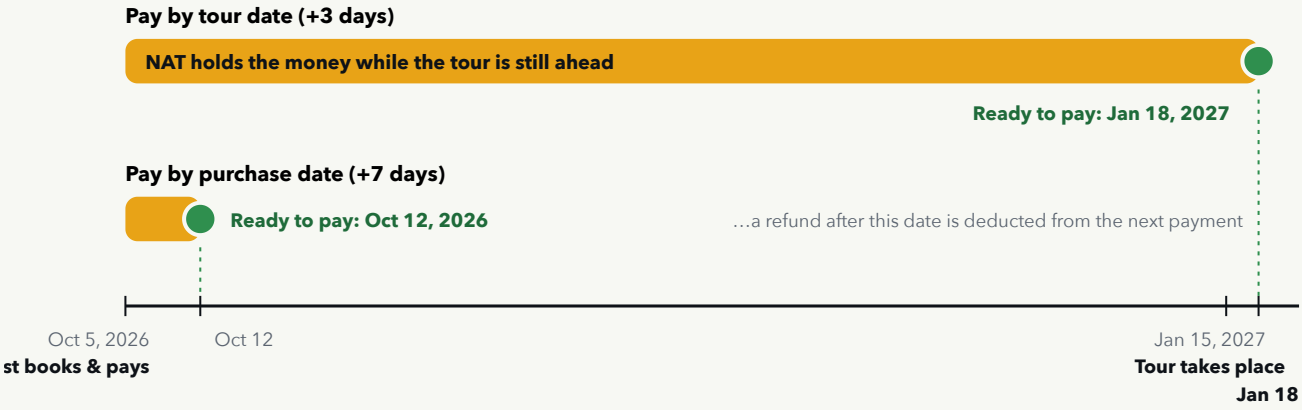


Fig. 3 – **EXAMPLE** The dates and day counts are illustrations to explain the two options; each operator's real values are set by NAT.

Shared rules for both options

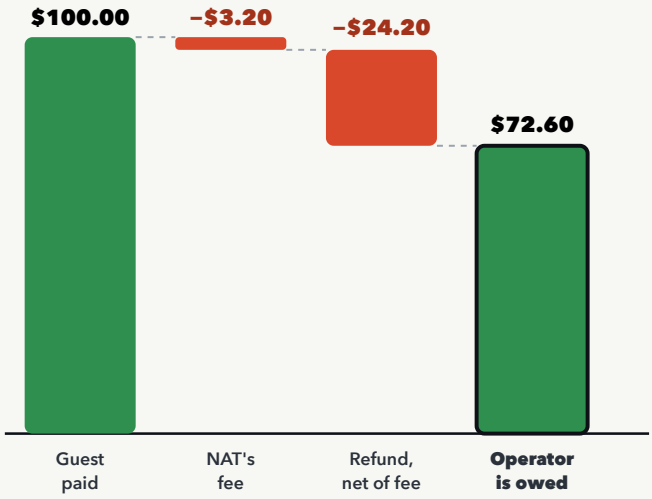
- The schedule (daily, weekly, monthly), minimum amount, maximum amount, start date and notification emails come from the operator's existing **Auto Payout rule** – nothing new to set up there.
- Only card payments are included. Cash, invoiced and tip payments are not.
- Payments that would need a refund-sized deduction larger than the next payment simply wait: the run is skipped and the amount carries forward until it is covered.
- If an operator's date setting is changed, it applies to payments not yet paid. Payments already paid are never re-opened.

05 — HOW THE AMOUNT IS WORKED OUT

One rule, applied to every payment

For each card payment: **operator is owed what the guest paid minus NAT's fee.** For each refund: the operator gives back what they would have received for it. Add them all up — that is the payment.

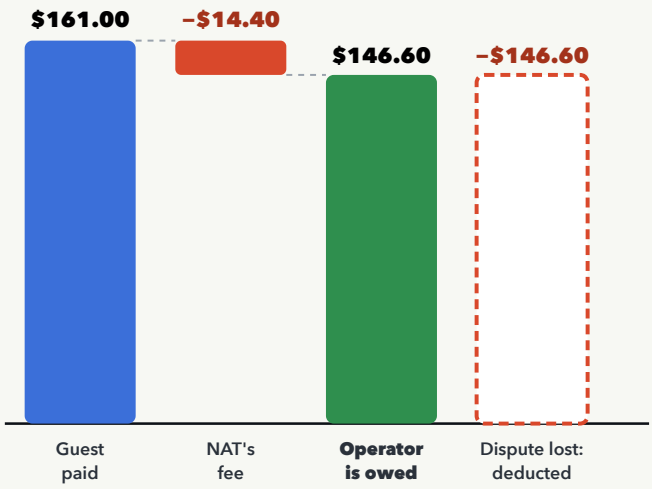
A booking with a refund



Refund $25.00 minus $0.80 of fee returned = $24.20

Fig. 4 — The worked example documented for today's Auto Payout. The new module must give the same $72.60.

A payment, then a lost dispute



NAT absorbs the $14.40 fee, exactly as it does today

Fig. 5 — The dispute example from today's dispute code: the operator is deducted only what they received.

The rules behind the numbers

- **Cents, not decimals.** All maths is done in whole cents, so a rounding error can't creep in.
- **One calculation, in one place.** Today's Auto Payout repeats its formula in three places, which is where mistakes hide. The new module has a single calculator that everything else uses.
- **Each payment on its own line.** Every payment and every refund is its own record. A refund after payment becomes a deduction on the next run, with no special "adjustment detection" needed.
- **Booked, then cancelled with no refund?** The operator keeps the money — same as Auto Payout today.

06 — EVERY SITUATION, IN PLAIN WORDS

"What happens if...?"

Each row below becomes an automated test, so the behaviour is checked every time the code changes.

SITUATION	WHAT HAPPENS
Normal booking paid by card	Becomes ready on the release date and is included in the next approved run.
Guest paid only part	Follows the same "include partial payments" choice Auto Payout uses today.
Extra people, add-ons, discounts, custom charges, switched package	Every payment is its own line, counted once. They can be paid in different runs without confusion.
Refund before the operator is paid	The payment and the refund cancel out. If nothing is left, no money moves for that booking.
Refund after the operator is paid	Deducted from the operator's next payment.
Deduction bigger than the next payment	No payment that time. The balance carries forward until it is covered.
Cancelled, "don't refund"	The operator keeps the money, as today.
Dispute lost	For operators with the Dispute Reversal setting: operator's share is deducted, NAT absorbs the fee. Disputes that are won or still open change nothing.
Booking split or moved	Each booking's total stays correct; no double counting.
Two currencies (USD, CAD)	Handled completely separately, each against its own balance.
Operator has no bank account / Stripe account not ready	The run shows ▲ **Needs attention** with a plain hint; nothing is lost.
Money not yet available in NAT's balance	The run shows ● **Waiting for funds** and retries. Never half-paid, never failed.
Tips and partner bookings	Tips are not part of this module — unchanged. A partner booking is paid like any other: the operator is owed payment − NAT fee (Section 10).

07 — HOW NAT STAYS PROTECTED

NAT keeps only its fee

Every dollar of ticket cost goes to the operator. **No safety deposit is held back and no minimum balance is kept aside** – NAT's balance keeps only NAT's own fee.

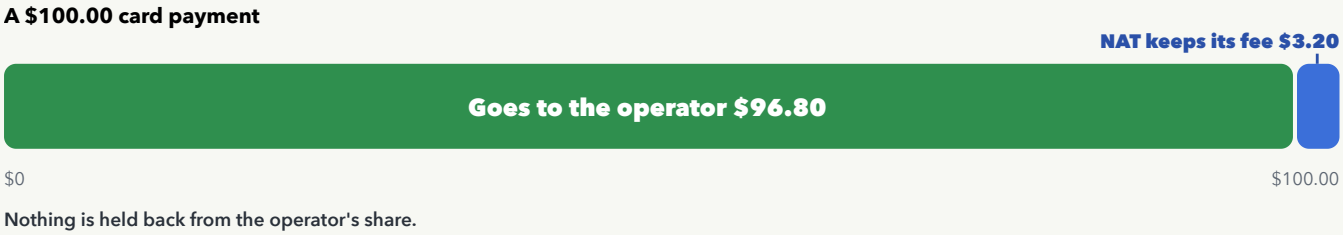


Fig. 6 – The payment and fee from the worked example in Section 5.

Four protections that don't hold anyone's money

1 **Later refunds come off the next payment**

If a guest is refunded after the operator was paid, that amount is deducted from the operator's next payment. If it is bigger than the next payment, it carries forward until it is covered.

2 **Lost disputes are deducted the same way**

For operators with the Dispute Reversal setting. NAT absorbs its own fee, exactly as it does today. Disputes that are won or still open change nothing.

3 **Money moves only when it is really available**

Recent card payments take a few days to clear in Stripe. Until then the run shows ● **Waiting for funds** and retries by itself. Nothing is ever half-paid. *(To be confirmed in Stripe test mode.)*

4 **A daily check compares what NAT holds with what NAT owes**

If NAT's balance would not cover what it owes operators, an alert goes out. It only warns – it never holds money back.

THE TRADE-OFF YOU CHOSE

With no safety hold, a refund or lost dispute that arrives *after* an operator was paid is covered by NAT first and recovered from that operator's next payment. If an operator stops taking bookings, there may be nothing left to recover from. A safety hold could be added later without redesigning anything.

08 — WHAT STAFF WILL SEE

Designed so anyone can follow it

The module is only useful if finance and support staff can read it without help. These are the design rules we will build to.

Plain words

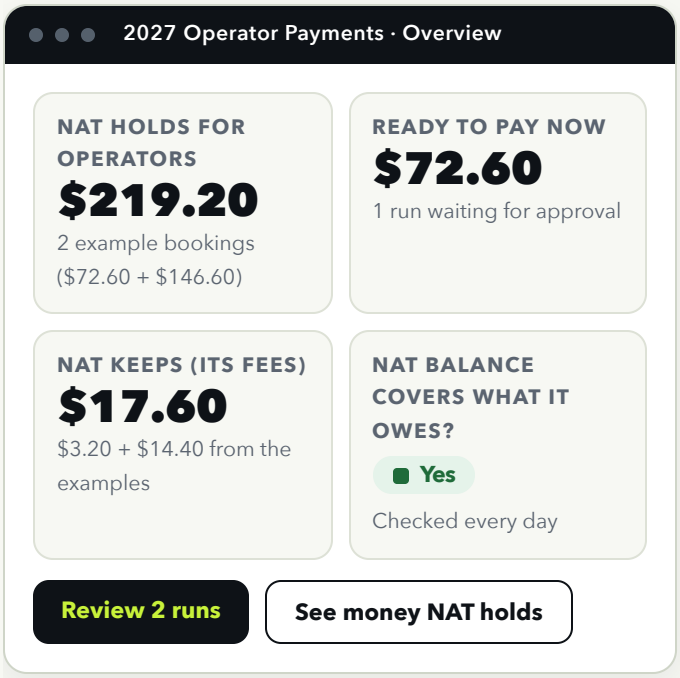
No "ledger", "transfer" or "idempotency" on screen. See the glossary below.

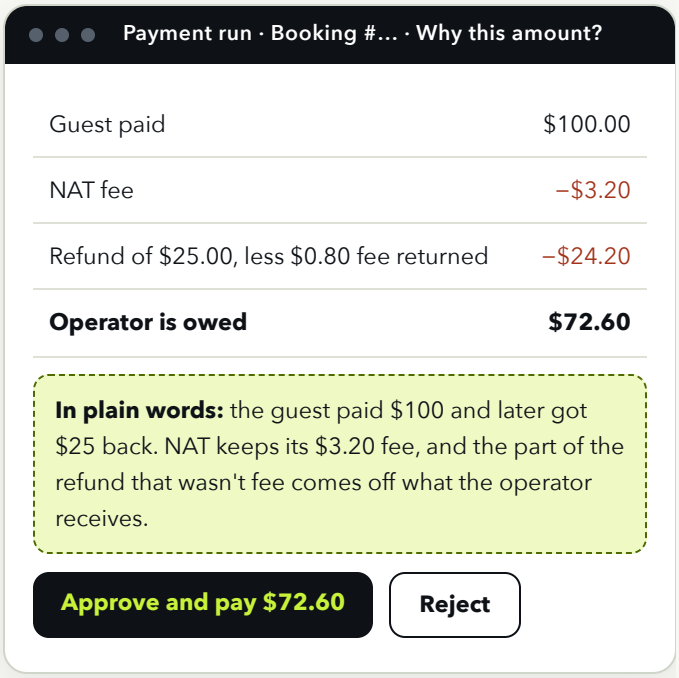
"Why this amount?"

One click on any figure shows the exact payments and refunds behind it.

Safe by default

Preview first. Buttons say what happens: "Approve and pay $X to Operator".





MOCK-UP Layout only. Both screens use the two worked examples from Section 5 ($72.60 and $146.60); the coverage indicator is schematic.

Words, messages and emails

When something needs attention, the message says what happened and what to do — never a code.

Glossary — the words we will use

TECHNICAL TERM	WHAT STAFF WILL SEE	MEANING
Batch / settlement	**Payment run**	One group of payments paid to one operator together.
Transfer	**Move to operator's Stripe account**	First half of paying: money leaves NAT's balance.
Payout	**Send to bank**	Second half: Stripe pays the operator's bank.
Adjustment / clawback	**Deduction from next payment**	Money taken back after a late refund or lost dispute.
Ledger	**Payment record**	The list proving each payment was paid once.

Messages written for people, not for developers

INSTEAD OF…	STAFF WILL READ…
`balance_insufficient`	The money for this payment isn't available in NAT's Stripe balance yet. We'll try again automatically — nothing for you to do.
`No bank account found for account acct_…`	This operator hasn't added a bank account in Stripe yet. Ask them to add one and the run will continue by itself.
`Payout amount exceeds maximum ceiling`	This run is above the maximum you set for this operator. Review it, or raise the limit and approve again.
`Payout failed`	The bank sent this payment back. The money is safe in the operator's Stripe account — check their bank details and we'll retry.

The email an operator receives MOCK-UP

Email · to the operator

Subject: Your payment of $72.60 for 2027 bookings is on its way

Hello — a guest paid $100.00 for your tour and later received a $25.00 refund. After NAT's $3.20 fee, **you are being paid $72.60**. It should reach your bank in a few business days.

Questions? Open the payment in your dashboard and choose "Why this amount?". *(Wording shows the tone; amounts come from the Section 5 example.)*

09 — HOW WE MAKE SURE THE NUMBERS ARE NEVER WRONG

Seven safeguards

Getting money wrong costs trust, so the design assumes mistakes will be attempted – and blocks them.

Each payment is paid once – enforced by the database

Every payment and refund gets one record that can exist only once. Even a software bug can't make the system pay the same payment twice.

The same answer as Auto Payout

A standing test feeds identical bookings to today's Auto Payout calculation and the new one and requires the result to match to the cent. A difference fails the build.

Preview mode before real money

For several cycles the module calculates and stores runs without touching Stripe. We compare them against an independent recalculation from Stripe's own payment and refund records.

A person approves every run (version 1)

Nothing moves without an admin's approval, a maximum amount per run can be set, and every action is recorded with who and when.

Safe to retry, never moves twice

Every Stripe action carries a unique key, and each step is remembered. A crash or timeout can be retried without any risk of a duplicate. A recovery tool finds any Stripe action that wasn't recorded.

A daily check: "What NAT holds vs. what NAT owes"

Every day the module confirms that every 2027 payment is accounted for exactly once, that each booking's total is right, and that Stripe's records match NAT's. It also shows whether NAT's balance covers what it owes operators, and alerts if not.

Test money first – production is not touched

Until you say go-live, only Stripe test (sandbox) keys are used. The module refuses to run with a live Stripe key unless a super admin has deliberately unlocked Live.

10 — PARTNER BOOKINGS

Paid the same way as any booking

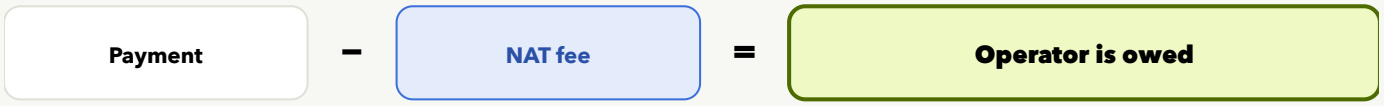
Partner (3-way) bookings split a payment between the operator and a partner. Another developer, Ketan, is changing this for 2027 on the branch `feature-partner-commision-simplification`.

WHAT THAT BRANCH DOES

- From 2027-01-01, for every operator, the partner's share is **no longer sent through Stripe**.
- It is only **recorded** in the payouts table, with an empty transfer id.

WHAT IT MEANS FOR OPERATOR PAYMENTS

- The operator is owed **payment − NAT fee** — nothing else is taken off.
- This module **does not deduct or pay** the partner's share.



The same rule for every booking, including partner bookings. Nothing else is subtracted.

DECIDED

For a partner booking the operator is owed *payment − NAT fee*, with nothing else deducted. Operator Payments does not pay the partner.

WORTH KNOWING

Because this module does not pay partners, the partner's share — recorded by Ketan's branch — has to be settled separately, outside Operator Payments. Ketan's deletion of `app/Services/UnitTest/UnitTest.php` (364 lines) has been confirmed as intended.

11 — SETTINGS AND SWITCHES

Who controls what

Wherever possible we reuse settings admins already know. Only a handful of new ones are added, each with a safe default.

SETTING	WHERE	WHAT IT DOES	DEFAULT
Operator Payments mode	Central settings	Off · Preview · Live – a single master switch for the whole platform.	Off
Stripe keys	Server settings (super admin)	Test (sandbox) keys only until go-live. The module refuses live keys unless a super admin has unlocked Live.	Test only
Pay by tour date or purchase date, and days	Per operator	When a payment becomes ready.	Tour date, same days as Auto Payout
Schedule, minimum, maximum, start date, emails	Existing Auto Payout rule	Reused as is – nothing new to configure.	As already set
Dispute Reversal	Existing per-operator setting	Decides whether a lost dispute is deducted from the operator.	As already set
2027 payout setting (existing)	Per operator, database only	Decides whether an operator's 2027 charges are held by NAT. **Stays protected.**	Off

Making the master switch safe to click

- Only a NAT super-admin permission can change it.
- **Live** needs a type-to-confirm step and is refused unless the most recent preview passed its checks.
- Until you say go-live, only Stripe **test keys** are used. On production, Live stays locked until a super admin unlocks it, and production Stripe is not touched before then.
- Every change is recorded automatically – who, when, old value, new value.
- Going back to **Off** or **Preview** is always allowed and takes effect immediately.

> **IMPORTANT ORDER OF EVENTS**
>
> The existing per-operator 2027 payout setting must **not** be switched on for any operator until Operator Payments is live. Otherwise money would sit with NAT with nothing to pay it out.

12 — THE BUILD PLAN

Seven phases, safest first

Each phase can be tested on its own. Only Stripe test money moves — and only from Phase 3. Nothing touches production until you say go-live.

0 **Decisions & data checks** — DONE

Decisions are made and a read-only survey of the development database is done (Section 13). Production is not touched.

1 **The calculator and payment records**

One calculation, the "paid once" records, the matching test against Auto Payout, and a preview command. No Stripe involved.

2 **Payment runs and approval screens**

Runs, approve/reject, the "Why this amount?" view, exports, plain-language statuses.

3 **Moving money and sending to the bank**

The two Stripe steps, retries, the available-funds check, webhooks, emails, activity log.

4 **Corrections and disputes**

Manual corrections and lost-dispute deductions, including carrying forward a deduction that is bigger than the next payment.

5 **Daily check and overview dashboard**

"What NAT holds vs. what NAT owes", alerts, the overview screen.

6 **Pilot**

Full run on the test operator in Stripe test (sandbox) mode. Going live on production is a separate step that only you decide.

FIXED ON OCTOBER 6 — CARD-STATEMENT NAME

For Canadian operators, 2027 card payments showed NAT's own name (`TOURS* ASWTA`) instead of the operator's. The payment gateway now sends the operator's name as the full statement line (`ASWTA`). Checked in Stripe test mode for online card payments and the card reader; Payment Links were already correct. One limit: a name shorter than 5 characters can't be used as a full line.

13 — WHAT THE DATA SAYS (PHASE 0)

What the survey found

A read-only survey of all 13 tenants in the **development** database. Nothing was changed. Production is not touched.

CHECK	RESULT	EFFECT ON THE PLAN
Packages with their own Stripe account	**gcw:** all 14 packages use one account that differs from the operator-level account. **ddmta:** 5 packages, 2 different accounts, 1 differs. The other 11 tenants match or have none.	The operator-level account for every tenant in version 1; the lookup is built so package accounts are a small later change.
Platform-held transactions	51 already exist — all on **kens**, where the 2027 payout setting is already on for testing.	Real test data for Phase 1.
Partner bookings	No package is flagged as a partner package today; 7 partner payment rows exist on kens.	Paid like any booking.
Lost disputes	None recorded in the last 52 weeks. Dispute Reversal is on for 9 of 13 tenants.	Deduction path is widely enabled.
Currencies	Every tenant reports USD — including **aswta**, which we have treated as Canadian.	Currency is read from live data, never assumed.
Auto Payout in use	Switched on for one tenant (kens), with no enabled rule.	Little live history to compare.

What this changes

- **One Stripe account per operator, with room to grow.** Every payment looks up its account through one small component. Today it always answers "the operator's account"; supporting package accounts later means changing only that component.
- **Phase 1 can be tested on real data.** The 51 platform-held transactions on kens let us run the calculator and the Auto Payout comparison before any Stripe work.

DECIDED

The operator-level Stripe account is the payee for **every** tenant. gcw and ddmta are mentioned only because they were the two tenants whose packages use a different account today; the rule is the same for everyone.

HOW WE TEST

Stripe **test (sandbox)** mode only. Production Stripe and the production database are not touched until you say go-live, and only a super admin can access production.

14 — DECISIONS AND NEXT STEPS

What is settled, and what is left

Everything is decided. The only thing needed now is your go-ahead to start building.

DECIDED

- Two deposits in version 1.
- Pay by tour date *or* purchase date, per operator.
- Refund after payment → deducted from the next payment.
- NAT keeps only its fee: no safety hold, no safety balance.
- Operator is owed payment − NAT fee − partner bookings too.
- Lost disputes only, via the existing setting.
- The operator-level Stripe account is the payee for every tenant; code ready for package accounts later.
- Master switch changeable in the app, with guard-rails.
- Stripe test (sandbox) mode only until go-live.

CONFIRMED ON OCTOBER 6

- **No safety hold and no safety balance.** Both are dropped; no customer money is held back.
- **Operator-level Stripe account** is the payee for every tenant.

Next steps

1. **You give the go-ahead** to start Phase 1.
2. **Phase 1 starts:** the calculator, the "paid once" records, and the test that compares it with Auto Payout – with no Stripe involved.
3. **Stripe test (sandbox) testing** begins in Phase 3, on the test operator only. Production is not touched.

NOT VERIFIED YET

This plan is based on reading the code. Stripe's behaviour (how fast moved money becomes available, what happens to a reversal after a bank payout) will be confirmed in Stripe's test (sandbox) mode during Phase 3.

APPENDIX — FOR DEVELOPERS

Technical notes

Skip this page if you are not building it. Everything above is the product; this is how it is anchored in the code.

New records

TABLE	PURPOSE
`settlement_batches`	One payment run per operator and Stripe account; status, totals, period, Stripe transfer and payout ids, retry state.
`settlement_items`	The payment record: one row per transaction, **UNIQUE(transaction_id)**; signed amount in cents; *owed-to* account; kind (payment, refund, adjustment, dispute).
`settlement_adjustments`	Manual corrections and lost-dispute deductions (credit/debit, pending/applied/voided).
`settlement_batch_logs`	Audit trail of every action; not cascade-deleted.

Naming avoids `payouts` (partner ledger) and `transfers` (used on the shelved deferred-transfer branch).

Key design points

- Per-row owed amount = ±(amount − fee_amount) in integer cents; one `SettlementCalculator`; parity test against `PayoutApprovalService::calculateAccountPayoutAmount`.
- Eligibility: Completed/Refund, `via='Card'`, `payment_mode='platform_held'`, not `request_from='Insert'`, no `#` comments; release = tour datetime or `created_at` + days (per-tenant setting). No lower date bound, so no "orphaned" bookings.
- Execution: global Redis lock → platform `Balance::retrieve` (available, per currency; transfer ≤ available, otherwise *Waiting for funds*) → `createTransfer` with idempotency key from batch reference → persist → bank payout (retry never re-transfers). No reserve or holdback. **Live-key guard:** refuse to run with a live Stripe key unless a super admin has unlocked Live.
- Account resolution through a `SettlementAccountResolver`: returns the operator's account in version 1; package-level accounts later mean changing only this class. Batches are grouped by account so that change needs nothing else.
- Disputes: hook `StripeWebhookController` lost-dispute path and manual mark-lost in `DisputeService::updateDisputeStatus`, gated by `DISPUTE_REVERSAL`, idempotent per dispute id.

Changes to existing code (small, but required)

- `StripeWebhookController::handleStripeAutoPayout`: match settlement payouts first; add the `payment_mode` filter to its fallback branch (today it would create a bogus Auto Payout record and set `payout_made=1`).
- `PayoutAdjustmentService` (3 detect methods): skip platform-held transactions, or refunds are counted twice.
- `PayoutApprovalService::recalculatePendingPayout`: add the `payment_mode` filter.
- `ProcessAutoPayouts`: run the settlement leg inside the same tick and lock; reconcile settlement payouts.
- Central `SettingController` whitelist + validation for the new keys; permission for the master switch.